

IMY 220 Project 2026

Photo Sharing Website

Deliverable 1

Instructions

- This deliverable must be completed by 07:30 on 4 September as indicated on ClickUP.
- You must demo your deliverable submission in the demo sessions in order to be awarded marks.
- You must demo your deliverable using Docker. Running the project locally will not be permitted.
- No late / email submissions will be accepted.

We recommend you constantly refer to the overall spec as well as your individual deliverable specs to make sure your project meets all of the requirements by the end.

Make sure you also continuously test your project as you add new functionality to make sure everything keeps working as expected.

Part 1: Basic React Project

Set up a basic React project using Vite, as used in the React practicals and assignments.

You do not need to wait for the later React/NodeJS lectures in order to get started. You already have enough knowledge of React components to begin creating the required files and structure.

While setting up your project, separate your frontend and backend code into different folders. You are required to use the following basic structure, but you may create additional folders where appropriate:

1. In the root of your project, create a folder called *frontend*. Your Vite React application must be created inside this folder.
2. In the root of your project, create a folder called *backend*. Place your `server.js` file in this folder. This is where your backend Express code will live. MongoDB functionality will be added in a later deliverable.
3. Inside `frontend/src`, create two subfolders called `components` and `pages` for your reusable components and page components respectively. Your main application entry files should remain inside `src`.

4. Store static assets such as images inside an appropriate assets folder within the frontend project.

Part 2: Frontend Components with Dummy Data

When it comes to components, you want to follow *separation of concerns*. Each component should be placed in its own file and should have a specific responsibility. You will need to decide what goes into each component and how you split up the functionality, but this section provides you with the bare minimum.

The components listed below represent the minimum required component structure. They do not describe all the content that must appear on each page. Refer to the main project specification for the full page requirements. You may add extra components if you wish. Components should follow the DRY (don't repeat yourself) principle and be reusable where appropriate. If a component can be reused in multiple places, you should reuse it rather than duplicate the JSX.

You should refer back to the main project specification to see the specific requirements for each page and that should guide you on what should be included in each component.

Your HTML should make use of semantic HTML elements (nav, main, aside, article, etc.) where appropriate. This is good practice.

General

- Post Preview component - Should encapsulate a post, would be useful for the feeds and profile pages.
- Create Post component - Should provide a way for a post to be created.
- Profile Preview component - Displays all basic information on a profile that is viewed in friend lists, search results, etc. - i.e., not the entire profile, but a small preview of it.
- Header / Navigation component – Contains the navigation links used to move between pages and must be present on all pages except the Splash page.

Splash Page

- Login Form - Contains all the information / inputs required to log in, stub out endpoints. [refer to part 5]
- Sign up Form - Contains all the information required / inputs to sign up, stub out endpoints. [refer to part 5]

Home Page (with Feed)

- Feed component - Displays the post preview components, this can be one component that handles the display of both feeds, or a separate component for each.

- Search Input component - Handles search inputs, the search functionality does not need to be implemented yet, you only need to be able to enter a search term based on your planned functionality.

Profile Page

- Profile component - Contains basic profile information.
- Component to edit a profile - Contains all the form information for editing the user's profile.
- A component to list all the posts that a user has created.
- Friend components - Displays all users that the current user is friends with.
- Component to create a post - Contains all the form information for adding a post.

Post Page

- Post component - Contains all required information on a single post.
- Image component - Displays the image for the post.
- Comments component - Displays all the comments left on a post.
- Component to edit a post - Contains all the form information for editing a post.

Part 3: Basic Functionality

The components created in Part 2 do not all need to have their final functionality implemented yet. Additional functionality will be added in later deliverables.

However, some components should have basic client-side functionality. This is to get you started on implementing some of the functionality you need before you are required to implement more extensive backend functionality in Deliverables 2 and 3.

Choose two of your form components you have created (i.e., any components that make use of HTML inputs) and implement basic client-side validation for them. This can either be done with JavaScript, HTML input validation ([MDN Docs](#)) or a mixture of the two.

- For example, checking if a value filled in is the correct number of characters or has a required symbol (e.g., @), checking if password and repeated passwords match, checking if input fields are not empty, etc.
- You may pick any of your form components, e.g., components with inputs to add / edit a piece of content, a login form, etc.
- You do not need to do any validation that requires a backend. However, you will be required to make use of either JavaScript (state) or HTML input

validation to ensure that the validation functions correctly (i.e., the form is not able to submit until all required fields are valid).

Part 4: Routing

Set up basic page routing to access each of your main pages via the URL. You do whatever method you prefer, but using *react-router* is advised.

- What is important is that a user is able to navigate to the Splash page, login / sign up forms (if they are not on the splash page), Home page, Profile page and Post page via the URL (i.e., `/`, `/home`, etc.). You may add any additional routing to any other pages you have if you wish to do so.
- Your Profile page and Post page should make use of dynamic routes. Eventually, you will need to be able to access a specific post or profile through its id in order to render the appropriate content using the same component template. This is done by passing an id through the URL. For now, you can render the same post or profile regardless of which id is passed through the URL, but you should be able to handle passing any id (e.g., `/profile/23532` or `/post/123`).

Part 5: Authentication Endpoints Stubbing

Stubbing is when you create endpoints that have no actual functionality, but rather return dummy data. Since you have no database yet, you can't actually make users and store them somewhere.

With this in mind, set up a basic Express server and create at least two stubbed authentication endpoints: one for sign-in and one for sign-up. You do not need to add any database functionality yet.

These endpoints do not need to connect to a database or perform real authentication. They should accept the appropriate request data and return suitable dummy response data.

Connect your sign-in and sign-up forms to these endpoints so that submitting the forms sends a request to the Express server and handles the returned response.

Persistent authentication and user identification will be implemented in a later deliverable.

Part 6: Docker

Create a Dockerfile and `.dockerignore` for both the frontend and backend of your application.

Your frontend and backend must each run in their own Docker container.

Include the commands required to build and run both containers in your README.txt file.

Your application must be fully functional when run using Docker. You should test your application thoroughly before Dockerising it. If something is not working in your local environment, it probably won't work in Docker. Once Dockerised, test both the frontend and backend containers thoroughly, as there can sometimes be differences between how operating systems and Docker environments handle certain functionality.

Important! You need to ensure that your application works using Docker. If it does not work, you won't be able to receive any marks on the marking day.

Do not wait until submitting to get your project working using Docker!

Submission Instructions

Place all your submission files in a folder named `Surname_D1`.

- Include all project files required to build and run your application, excluding *node_modules*.
- Include all files required to build and run both Docker containers.
- Include a README.txt file containing:
 - the commands required to build and run both Docker images/containers;
 - a link to your GitHub repository.
- At this point, you should have two additional commits. These may be committed to your main branch.

Zip the folder into a ZIP file named *Surname_D1.zip* and upload it to the relevant submission slot on ClickUP before the deadline.
